

INF6953QE - Internet of Things Security

Practical Work 2 – Fall 2026

Federated Learning in IoT Security

Instructions:

- This work is to be submitted via Moodle only.
- The submitted report must be in PDF format. You're free to build it in any format you want, however (.docx, .odt, .tex, etc.).
- The report must contain a title page including the course title, the lab title, your names, and student ID numbers (matricule).
- The report is limited to a maximum of 5 pages. The code should be submitted as a separate file (e.g., .py or .ipynb) and will not count toward the page limit.
- The report must be submitted by the 3rd of October 2026 before 23h59. A penalty of 10% will be applied for each day after that date.
- There are 4 labs in total, each worth 14 marks/7% weighting (56 marks/28% total).

Background and Introduction:

Background: Federated Learning (FL) is a decentralized machine learning paradigm that has gained particular traction in IoT environments, where protecting data privacy is a central concern. Rather than pooling raw data in one location, FL keeps data on individual devices and only exchanges model updates, allowing multiple devices to jointly train a model without ever exposing their underlying data (McMahan et al., 2017).

Conventional machine learning pipelines typically require centralizing data from all participating sources, which raises significant privacy risks. FL addresses this by enabling devices to contribute to a shared, globally trained model while keeping their raw data local (Bonawitz et al., 2019). In this lab, you will build a simplified FL pipeline within a simulated IoT network, implement the aggregation of locally trained models, and assess how well the resulting global model performs.

Topology Instructions:

- Simulate a minimum of 5 IoT devices, though you're encouraged to go beyond this. Testing with 7 or more devices will let you observe how performance shifts as the topology scales.
- Split the chosen IoT dataset into separate local partitions, one per simulated device. Introduce some variation across these partitions (different samples, feature distributions, etc.) so each device reflects a realistic, non-identical data profile.
- Jupyter Notebook is sufficient to represent this setup (no physical hardware is required). Each "device" can simply be modeled as its own dataset partition paired with its own local training routine.

Dataset: <https://www.unb.ca/cic/datasets/iotdataset-2023.html>

Question 1: Setting Up a Federated Learning Environment (3)

To simulate a Federated Learning setup in an IoT security context, you'll create a central aggregation server alongside several simulated IoT devices, each responsible for training a model on its own local data. Start by building a Python-based simulation (in Jupyter Notebook or a comparable environment) where every device is assigned a distinct partition of an IoT dataset (e.g., network traffic, sensor readings, or intrusion records). Each device should then independently train a lightweight classifier, such as logistic regression, on its assigned partition.

Set up your environment with the required libraries: Flower, pandas, numpy, scikit-learn, and Flask (the latter for simulating inter-device communication). Use `LogisticRegression` from `sklearn.linear_model` as your local model. Make sure each device's dataset partition is distinct in content while maintaining a consistent structure across devices. Once trained, each device should transmit its model weights to the central server.

Steps:

- Import the required libraries in Jupiter Notebook.
- Load a real IoT dataset (CSV or other format) and split it into per-device partitions.
- Train a local logistic regression model on each device's data.
- Have each device forward only its model parameters (never the raw data) to the central server for aggregation.

Question 2: Aggregating Model Parameters on the Server (3)

After each device has completed local training, the central server's job is to combine these individually trained models into a single global model. The standard approach in Federated Learning is to average the local model weights. Your simulation should include a server-side function that gathers the parameters submitted by each device and computes this average.

Within your notebook, implement a function that pulls in the weight vectors from every participating device, averages them, and uses the result to update the global model. Once aggregation is complete, the updated global model should be sent back out to the devices so they can continue training on it.

Steps:

- Write a function on the server side to collect incoming model parameters from each device.
- Compute the global model by averaging the collected weight vectors.
- Apply the averaged result to update the global model.
- Redistribute the updated global model to all devices.

Question 3: Evaluating Federated Learning Model Performance (4)

Once your global model has been trained, its performance needs to be validated on data that the individual devices did not train on. Use `train_test_split` from `sklearn.model_selection` to carve out a held-out test set from the IoT dataset before any device-level training begins; this set should remain untouched by the local training process.

Report accuracy, precision, and recall for the aggregated global model, and benchmark these against a model trained the traditional way, on centralized data. This comparison surfaces the accuracy-versus-privacy trade-off that's central to FL. Use `matplotlib` or `seaborn` to visualize how these metrics evolve across training rounds.

Steps:

- Evaluate the trained global model against the held-out test set.
- Train a comparable centralized model and compare its performance to the global model's.
- Plot the performance metrics over training iterations using `matplotlib` or `seaborn`.

Question 4: Simulating Adversarial Attacks and Defense Mechanisms in Federated Learning (4)

This step introduces a malicious participant into your FL setup. This device deliberately sends corrupted or skewed updates instead of legitimate ones. Implement this by tampering with the model parameters on that device before transmission, then observe the resulting impact on the global model's performance.

Next, implement at least one defense mechanism to counter this behavior:

- **Robust Aggregation:** Replace simple averaging with a more resilient method, such as the median or a trimmed mean of the submitted weights.
- **Anomaly Detection:** Flag updates that deviate significantly from the norm and exclude them from the aggregation step.

Compare the global model's performance with and without the defense in place to assess its effectiveness.

Steps:

- Introduce a malicious device that submits manipulated model updates.
- Implement a defense mechanism (robust aggregation, anomaly detection, or both).
- Quantify the attack's impact and the defense's effectiveness by comparing performance before and after mitigation.

Question 5 (Bonus): Comparing Machine Learning Models in Federated Learning (+2)

Beyond logistic regression, implement one additional lightweight classifier and compare its federated performance to your logistic regression baseline.

Steps:

- Implement one additional model (Decision Tree, SVM, or Naïve Bayes) alongside logistic regression.
- Train both models using the same federated setup and compare their aggregated performance (accuracy, precision, recall, F1-score).
- Briefly discuss which model performed better and why, considering accuracy vs. computational cost.

Resources:

- <https://flower.ai/>
- <https://www.unb.ca/cic/datasets/iotdataset-2023.html>
- <https://dataplatfom.cloud.ibm.com/docs/content/wsj/analyze-data/fl-tf2-samples.html?context=cpdaas>
- <https://dataplatfom.cloud.ibm.com/docs/content/wsj/analyze-data/fldemo.html?context=cpdaas>
- <https://medium.com/@adam.narozniak/federated-learning-with-flower-and-tensorflow-d39e2d04f551>
- <https://www.restack.io/p/federated-learning-answer-jupyter-notebook-cat-ai>

References:

1. McMahan, B., Moore, E., Ramage, D., Hampson, S., & y Arcas, B. A. (2017). Communication-efficient learning of deep networks from decentralized data. In *Artificial Intelligence and Statistics* (pp. 1273–1282). PMLR.
2. Bonawitz, K., Eichner, H., Grieskamp, W., Huba, D., Ingerman, A., Ivanov, V., ... & Van Overveldt, T. (2019). Towards federated learning at scale: System design. *Proceedings of the 2nd SysML Conference*.
3. Neto, E. C. P., Dadkhah, S., Ferreira, R., Zohourian, A., Lu, R., & Ghorbani, A. A. (2023). CICIOT2023: A Real-Time Dataset and Benchmark for Large-Scale Attacks in IoT Environment. *Sensors*, 23(13), 5941.